Problema dos Nós Críticos

Leonardo M. Machado[1], **Samuel M. da Rocha e Silva**[1]

[1]Departamento de Informática e Matemática Aplicada –
Universidade Federal do Rio Grande do Norte (UFRN)
59.078-970 – Natal – RN – Brazil

Abstract. *The Critical Node Problem (CNP) seeks a set of K vertices whose removal minimizes the pairwise connectivity of the residual graph. This work reports the implementation, in Python, of two versions of the greedy constructive heuristic Greedy2: a naive one, which evaluates $f(S \cup \{i\})$ from scratch for each candidate, and an optimized one, which uses the EVALUATE algorithm to compute the impact of all vertices in a single DFS. We analyze the time complexity of both and apply them to 16 benchmark instances. Both versions produce identical solutions, with the optimized one being 19 to 1496 times faster. A comparison with 8 state-of-the-art algorithms allows the analysis of the results.*

Resumo. *O Problema dos Nós Críticos (CNP) busca um conjunto de K vértices cuja remoção minimiza a conectividade par-a-par do grafo residual. Este trabalho relata a implementação, em Python, de duas versões da heurística construtiva gulosa Greedy2: uma ingênua, que avalia $f(S \cup \{i\})$ do zero para cada candidato, e uma otimizada, que emprega o algoritmo EVALUATE para calcular o impacto de todos os vértices em uma única DFS. Analisamos a complexidade temporal de ambas e as aplicamos a 16 instâncias de benchmark. As duas versões produzem soluções idênticas, com a otimizada sendo de 19 a 1496 vezes mais rápida. Foi feita a comparação com 8 algoritmos do estado da arte, que viabiliza a análise dos resultados.*

1. Introdução

O Problema dos Nós Críticos - o CNP - é um problema de otimização focado na busca por um conjunto de nós em um grafo cuja remoção degrade ao máximo alguma métrica de conectividade (Lalou et al. 2018). Encontrar tal conjunto de nós significa apontar os vértices mais importantes para que uma rede de elementos mantenha-se resiliente apesar de partições.

Em vista das múltiplas aplicações deste problema em diversas áreas de conhecimento, este trabalho propõe-se a: **relatar a implementação de duas versões de uma heurística construtiva gulosa já apresentada pela literatura, uma versão ingênua e uma otimizada**, **detalhar a complexidade temporal das soluções** e **expor os resultados obtidos sobre a ótica do melhor valor observado para a função objetivo e do desempenho da solução, assim comno compará-los a resultados existentes**.

2. Descrição do Problema

Seja $G = (V, E)$ um grafo, tal que V é o conjunto de vértices e E, de arestas. O CNP consiste em, dado como parâmetro um inteiro arbitrário K, com $K \leq |V|$, encontrar o conjunto $S \subseteq V$ com $|S| = K$ que minimize valor de $f(S)$, sendo S o conjunto de

vértices removidos do grafo residual $G[V \setminus S]$ e f a função objetivo que calcula uma métrica de conectividade do grafo (Arulselvan et al. 2009). Esta métrica, como disposto por Lalou et al. 2018, substancia a definição de um dado nó como *crítico*, e afeta diretamente os procedimentos de otimização.

Para o caso deste trabalho, a função objetivo aplicada será a contabilização da conectividade par-a-par do grafo, definida por Ventresca and Aleman 2015 e Zhou et al. 2019 como: $f(S) = \sum_{C_i} \binom{|C_i|}{2}$; sendo C_i os componentes conexos pertencentes ao grafo residual $G[V \setminus S]$. O cálculo da combinação binária do tamanho de um dado componente retorna o número de conexões presentes entre dois de seus vértices, e o somatório totaliza as conexões mantidas em $G[V \setminus S]$ (Ventresca and Aleman 2015, Zhou et al. 2019). Essa definição se alinha conceitualmente à descrição usada por Addis et al. 2016, com a função objetivo calculando o número de pares de vértices ainda conectados no grafo residual $G[V \setminus S]$.

3. Complexidade do problema

Arulselvan et al. 2009 apresenta a demonstração de que a versão de decisão do CNP - chamada de K-CNP - pertence à classe $\mathcal{NP}$-completa, e que, consequentemente, o problema de otimização CNP pertence à classe $\mathcal{NP}$-difícil.

A definição do K-CNP, segundo Arulselvan et al. 2009 é:

> **K-CNP.** Dados um grafo não direcionado $G = (V, E)$ e um inteiro k, existe um conjunto $S \subseteq V$ com $|S| \leq k$ tal que $f(S) \leq k$, onde f denota a conectividade par-a-par do grafo residual?

A partir deste problema, é feita a demonstração de que K-CNP $\in \mathcal{NP}$-completo com dois passos lógicos: (i) mostrar que K-CNP $\in \mathcal{NP}$, apresentando um algoritmo de verificação em tempo polinomial; e (ii) mostrar uma redução $\leq_\alpha$ do problema $\mathcal{NP}$-completo Independent Set Problem (ISP) ao K-CNP.

A $\mathcal{NP}$-dificuldade do CNP é então mostrada pela redução do K-CNP. Dado que uma mesma instância (G, k) é a entrada para ambos os problemas e que a solução ótima do CNP é exatamente a pergunta respondida pelo K-CNP, caso exista um algoritmo que resolva o CNP retornando o conjunto de vértices removidos S em tempo polinomial, podemos resolver também K-CNP em tempo polinomial executando se $f(S) \leq k$.

Portanto, a $\mathcal{NP}$-completude de K-CNP prova a $\mathcal{NP}$-dificuldade do CNP.

4. Heurísticas construtivas e algoritmos aproximativos existentes

4.1. Heurísticas construtivas

Addis et al. 2016 propõem, além da heurística construtiva *Greedy2*, na qual este trabalho foca, outras três heurísticas:

- *Greedy1*: baseado em Arulselvan et al. 2009, parte de uma cobertura de vértices S, ou seja, um conjunto que, ao ser removido, não deixa nenhuma aresta no grafo residual $G[V \setminus S]$ (logo, $f(S) = 0$). Enquanto $|S| > K$, toma um vértice de S e o adiciona de volta a $G[V \setminus S]$, selecionando aquele que causa o menor aumento no valor de $f(S)$. A desvantagem clara é a dependência da cobertura inicial, que impede a análise de um vértice não contemplado nela.

- *Greedy2*: a heurística implementada neste trabalho, detalhada na Seção 7.1.
- *Greedy3d* e *Greedy4d*: heurísticas mais complexas que combinam a estratégia de remoção do *Greedy1* com a de adição do *Greedy2*, alternando entre as duas. O *Greedy3d* parte de uma cobertura de vértices, e o *Greedy4d*, do conjunto vazio. Ambas aplicam reinícios dinâmicos caso, em uma quantidade I de iterações, não seja obtida melhora no valor de $f(S)$.

Destacamos também outras duas heurísticas relacionadas a este trabalho. A primeira é a heurística original de Arulselvan et al. 2009, que inspirou o *Greedy1*: ela parte de um conjunto independente maximal (cujo complemento é uma cobertura de vértices), devolve vértices ao grafo de forma gulosa até restarem K vértices removidos e, por fim, melhora a solução com uma busca local. A segunda é o guloso de Ventresca and Aleman 2015, que segue a mesma estratégia do *Greedy2*, mas avalia todos os candidatos de uma iteração com uma única busca em profundidade. Esta é a otimização implementada neste trabalho, detalhada na Seção 7.3.

4.2. Algoritmos aproximativos

Um algoritmo aproximativo é aquele que, em tempo polinomial, encontra uma solução com a garantia de que ela não está muito distante da ótima, por exemplo, com valor de f no máximo α vezes o valor ótimo, para algum fator α fixo.

Para o CNP, esse tipo de garantia não é possível, a menos que $\mathcal{P} = \mathcal{NP}$. Isso acontece porque, em alguns grafos, é possível remover K vértices de forma que não sobre nenhum par conectado, ou seja, o valor ótimo é $f = 0$. Nesse caso, um algoritmo com garantia α teria que encontrar exatamente essa solução, já que $\alpha \cdot 0 = 0$. Porém, descobrir se essa solução existe é o mesmo que descobrir se o grafo possui uma cobertura de vértices com no máximo K vértices, que é um problema $\mathcal{NP}$-completo.

Por isso, os trabalhos encontrados na literatura apresentam garantias mais fracas:

- Ventresca and Aleman 2014 propõem um algoritmo de tempo linear, que resolve uma versão simplificada (relaxada) do problema e a converte em uma solução para o CNP, com um limite para a qualidade média da solução obtida.
- Dinh et al. 2012 estudam o problema "ao contrário": em vez de fixar K e minimizar a conectividade, fixam o quanto a conectividade deve diminuir e buscam o menor número de vértices a serem removidos. Para essa versão do problema, apresentam um algoritmo com garantia aproximada.

O *Greedy2*, implementado neste trabalho, não possui garantia de aproximação. Por isso, avaliamos a qualidade de suas soluções comparando-as com os resultados de outros algoritmos da literatura, na Seção 9.

5. Banco de instâncias

5.1. Grafos de entrada

Para a execução experimental deste trabalho, foi selecionado o conjunto de grafos apresentado por Ventresca 2012, disponível publicamente em ⟨http://www.individual.utoronto.ca/mventresca/cnd.html⟩ (acesso em: 12 out. 2026).

O conjunto compreende 16 grafos sintéticos, com número de vértices e arestas variados, construídos a partir de quatro modelos geradores de redes: Erdős–Rényi (ER),

Barabási–Albert (BA), Watts–Strogatz (WS) e Forest Fire (FF). Os arquivos estão no formato de lista de adjacências: a primeira linha contém o número N de vértices, e cada uma das N linhas seguintes contém o identificador do vértice e a lista de seus vizinhos. Esse formato facilita a implementação dos grafos enquanto listas de adjacências, que foi o modelo adotado neste trabalho.

Apontamos que a nomeclatura dos grafos de entrada podem variar. Por exemplo, Zhou et al. 2019 nomeia cada grafo a partir das iniciais do nome do modelo gerador utilizado e o número de vértices do grafo resultante, isto faz o grafo referenciado neste trabalho como *ErdosRenyi_n250* ser nomeado *ER235*, embora tenham o mesmo conteúdo. Optamos, neste trabalho, por preservar a nomenclatura definida no endereço do qual os grafos foram extraídos.

Esse conjunto de grafos é utilizado em diversos trabalhos da literatura, incluindo Addis et al. 2016, Zhou et al. 2019 e Ventresca and Aleman 2015, o que permite comparação direta com resultados publicados.

5.2. Instâncias

No Problema dos Nós Críticos, cada instância é definida como um par (grafo de entrada, valor de K), onde K é o número de vértices a serem removidos. Como o valor de K altera o conjunto de soluções viáveis e o valor ótimo da função objetivo, cada par (grafo, K) constitui uma instância distinta do problema.

Os valores de K utilizados neste trabalho foram extraídos da Tabela 4 de Zhou et al. 2019. Essa tabela agrega, para cada instância definida por um grafo de um conjunto que inclui as entradas propostas por Ventresca 2012 e um valor fixo de K, o melhor valor conhecido da função objetivo (*Known Best Value*, KBV) e os resultados obtidos por oito algoritmos do estado da arte, sendo estes:

- **Heurísticas construtivas gulosas**: *Greedy3d*, *Greedy4d*, *CNA1* e *FastCNP*;
- **Metaheurísticas**: *Best VNS*, *Best ILS*, *GA* e *MACNP*.

A seleção dos valores de K deste conjunto viabilizam a comparação direta dos resultados.

A Tabela 1 resume as 16 instâncias consideradas neste trabalho, com seus respectivos valores de K vistos em Zhou et al. 2019 e os tamanhos dos grafos.

6. Links importantes

- Repositório com a implementação, as instâncias, os resultados e as instruções de execução: ⟨https://github.com/SamuelMoraisdRS/PAA-CNP⟩
- Instâncias de Ventresca 2012: ⟨http://www.individual.utoronto.ca/mventresca/cnd.html⟩
- Versão aberta de Zhou et al. 2019, com a Tabela 4 usada nas comparações: ⟨https://arxiv.org/abs/1705.04119⟩
- Versão aberta de Addis et al. 2016: ⟨https://optimization-online.org/2015/01/4764/⟩
- Artigo de Ventresca and Aleman 2015: ⟨https://doi.org/10.1186/s40649-015-0010-y⟩

Tabela 1. Instâncias utilizadas neste trabalho.

Nome	$\|V\|$	$\|E\|$	K
ErdosRenyi_n250	235	350	50
ErdosRenyi_n500	466	700	80
ErdosRenyi_n1000	941	1.400	140
ErdosRenyi_n2500	2.344	3.500	200
BarabasiAlbert_n500m1	500	499	50
BarabasiAlbert_n1000m1	1.000	999	75
BarabasiAlbert_n2500m1	2.500	2.499	100
BarabasiAlbert_n5000m1	5.000	4.999	150
WattsStrogatz_n250	250	1.246	70
WattsStrogatz_n500	500	1.496	125
WattsStrogatz_n1000	1.000	4.996	200
WattsStrogatz_n1500	1.500	4.498	265
ForestFire_n250	250	514	50
ForestFire_n500	500	828	110
ForestFire_n1000	1.000	1.817	150
ForestFire_n2000	2.000	3.413	200

7. Implementação de heurística construtiva

A heurística construtiva implementada baseia-se no algoritmo *Greedy2* apresentado por Addis et al. 2016.

A versão contemplada pelo pseudocódigo é ingênua e de implementação relativamente simples. Por fins de otimização, foi também desenvolvida a modificação proposta por Ventresca and Aleman 2015, neste trabalho chamada de *Greedy2_dfs*.

7.1. Heurística Construtiva *Greedy2*

O *Greedy2* de Addis et al. 2016 constrói a solução S iterativamente a partir do conjunto S vazio. A cada passo, para cada vértice $i \notin S$, calcula-se $f(S \cup \{i\})$ e escolhe-se o vértice que minimiza esse valor. O processo se repete até que $|S| = K$. O Algoritmo 1 apresenta o pseudocódigo conforme a notação original dos autores.

Algorithm 1 Pseudocódigo do algoritmo *Greedy2*.

Require: Grafo $G = (V, E)$, inteiro K
Ensure: Conjunto $S \subseteq V$ com $|S| = K$
1: $S \leftarrow \emptyset$
2: **while** $|S| < K$ **do**
3: $\quad i \leftarrow \arg\max\{f(S) - f(S \cup \{i\})\}$
4: $\quad S \leftarrow S \cup \{i\}$
5: **end while**
6: **return** S

Fonte: Adaptado de Addis et al. 2016.

7.2. Implementação do *Greedy2*

O processo de implementação do algoritmo de Addis et al. 2016 envolveu adaptações e mudanças para fins de simplicidade, legibilidade e conformidade com a linguagem de programação selecionado. Em função disto, e prezando pela reprodutibilidade deste trabalho, apresentamos Algoritmo 3 e Algoritmo 2 como as versões efetivamente implementadas dos algoritmos *Greedy2* e do cálculo de $f(S)$, respectivamente.

7.2.1. Função objetivo

Propomos uma implementação de autoria própria para a função objetivo f, a partir da definição apresentada na Seção 2. A função recebe como parâmetros o grafo de entrada $G = (V, E)$ e o conjunto de exclusão $S \subseteq V$, e retorna a conectividade par-a-par do grafo residual $G[V \setminus S]$, ou seja, o número de pares de vértices ainda conectados por um caminho em $G[V \setminus S]$.

Para identificar os componentes conexos de $G[V \setminus S]$, realiza-se uma busca em largura (BFS) percorrendo os vértices $v \in V$ tais que (i) $v \notin S$ e (ii) v ainda não foi marcado como visitado, evitando contabilizar o mesmo componente múltiplas vezes. Para cada componente C identificado pela BFS, uma variável acumuladora é incrementada com o coeficiente binomial $\binom{|C|}{2}$, que corresponde ao número de pares de vértices internos ao componente.

A escolha da busca em largura ao invés de uma busca em profundidade foi feita pela intuitividade da implementação iterativa da BFS, diferente da DFS, que mais se adapta a versões recursivas. Como dito por Ventresca and Aleman 2015 sobre a aplicação de algoritmos de busca, a recursividade deve ser evitada pela possibilidade de estouro de pilha ao tomar grafos maiores.

7.2.2. Complexidade assintótica de f

A complexidade temporal de pior caso para o Algoritmo 2 é definida em função da busca em largura performada no grafo $G[V \setminus S]$.

A inicialização dos vetores auxiliares possuem custo da ordem $\Theta(|V|)$; e a marcação de vértices de S como visitados tem custo $\Theta(|S|) \leq O(|V|)$. O laço externo (linha 7 do Algoritmo 2) percorre todos os vértices de G, executando $|V|$ iterações com custo $\Theta(|V|)$.

Para analisar o processamento interno da BFS - considerando as manipulações na fila Q - denotamos que a busca é iniciada no primeiro vértice $v \in V$ ainda não visitado de cada componente, ou seja, uma vez por componente conexo de $G[V \setminus S]$. Sendo T o conjunto de componentes conexos de $G[V \setminus S]$, tem-se que $|T| \leq |V|$, portanto, o custo desta etapa é $O(|V|)$.

Os dois laços - *while* e *for* - compartilham um estado, sendo este o vetor marcando os vértices como *visitados*, que decresce a cada iteração. Deve-se contabilizar os custos de forma aditiva.

O laço *while* da BFS é executado uma vez para cada vértice desinfileirado. Já que

Algorithm 2 Função objetivo $f(S)$: conectividade par a par do grafo residual.

Require: Grafo $G = (V, E)$, conjunto $S \subseteq V$ de vértices removidos
Ensure: $f(S) = \sum_{C_i} \binom{|C_i|}{2}$, onde C_i são os componentes conexos de $G[V \setminus S]$
visitado $\leftarrow$ [FALSO] $\times |V|$ ▷ vetor de marcação
for $v \in S$ **do**
    visitado$[v] \leftarrow$ VERDADEIRO ▷ marca removidos como visitados
end for
total $\leftarrow 0$
for *fonte* $\leftarrow 0$ **até** $|V| - 1$ **do**
    if *visitado*[*fonte*] **then**
        continue ▷ já visitado ou removido
    end if
    visitado[*fonte*] $\leftarrow$ VERDADEIRO
    $Q \leftarrow$ FILA(*fonte*) ▷ inicia BFS
    tamanho $\leftarrow 0$
    while $Q \neq \emptyset$ **do**
        $u \leftarrow$ DESENFILEIRA(Q)
        tamanho $\leftarrow$ *tamanho* $+ 1$
        for $w \in G.$*adj_list*$[u]$ **do**
            if $\neg$*visitado*$[w]$ **then**
                visitado$[w] \leftarrow$ VERDADEIRO
                ENFILEIRA(Q, w)
            end if
        end for
    end while
    total $\leftarrow$ *total* $+ \binom{\textit{tamanho}}{2}$ ▷ soma os pares do componente
end for
return *total*

Implementação de nossa autoria, baseada na definição de $f(S)$ de Addis et al. 2016 e Ventresca and Aleman 2015.

cada vértice pode ser enfileirado e desinfileirado apenas uma vez, o custo será no máximo $\Theta(|V|)$, logo, $O(|V|)$.

Para o laço *for* iniciado na linha 17, temos o enfileiramento dos vértices que ainda não foram marcados como visitados. Este *loop* será executado $deg(u)$ vezes, sendo deg a função que retorna o grau do vértice desinfileirado u. Considerando todo vértice $u \in V \setminus S$, temos o número total de iterações dado pelo somátorio $\sum_{u \in V \setminus S} deg(u) \leq \sum_{u \in V} deg(u)$.

Já que cada aresta do grafo é contabilizada duas vezes, uma para cada vértice, temos que: $\sum_{u \in V} deg(u) = 2|E|$, logo, $\sum_{u \in V \setminus S} deg(u) \leq 2|E| \leq O(|E|)$. Portanto, por toda a execução da BFS, teremos custo $O(|E|)$ para esta etapa.

Somando os custos obtidos, chegamos à complexidade geral da solução com ordem $O(|V| + |E|)$.

7.2.3. Função principal do *Greedy2*

A implementação no Algoritmo 3 reproduz a semântica descrita em Algoritmo 1.

A chamada de função $\arg\max \{f(S) - f(S \cup \{i\})\}$ é desdobrada em uma estrutura de repetição interna que itera pelos vértices $i \in V$ buscando o nó cuja adição à S leve ao menor valor $f(G, S \cup \{i\}))$. Tal nó é então selecionado pela heurísitica gulosa como *crítico* e adicionado ao conjunto de remoção.

7.2.4. Complexidade assintótica do *Greedy2*

O *loop* principal será executado $\Theta(K)$ vezes, com $K \leq |V|$. Enquanto o laço interno será executado até $|V|$ vezes com chamadas à função f para cada vértice $i \notin S$. Como visto na Seção 7.2.2, f apresenta complexidade $O(|V| + |E|)$. Agregamos, portanto, a complexidade de pior caso $O(K \cdot |V| \cdot (|V| + |E|))$. Já que $K \leq |V|$, reduzimos a expressão para $O(|V|^3 + |V|^2|E|)$.

7.3. Implementação do *Greedy2_dfs*

Em vista da complexidade temporal mostrada na Seção 7.2.3, surge o interesse de buscar otimizações que viabilizem a execução da heurística construtivas em grafos de tamanho expressivo. Para sanar esta necessidade, foi implementado a otmização à abordagem gulosa proposta por Ventresca and Aleman 2015 por meio da substituição da função objetivo f pelo algoritmo *EVALUATE*.

A otimização desta abordagem baseia-se em um conceito simples, ao invés de calcular a configuração de S que minimiza $f(S)$ de forma iterativa e repetida para cada iteração - como feito no Algoritmo 3, Ventresca and Aleman 2015 propõe calcular, em uma única passada, os efeitos na função objetivo ao remover qualquer vértice $v \in V$ a partir de uma função $g(v)$ e disponibilizar estes valores para acesso no algoritmo principal.

A intuição para o algoritmo é descrita com rigor formal e profundidade por

Algorithm 3 Heurística construtiva gulosa Greedy2 (implementação).

Require: Grafo $G = (V, E)$, inteiro K

Ensure: Lista *ordem* com os vértices removidos na ordem de escolha e lista *historico* com $f(S)$ após cada iteração

```
1:  S ← ∅
2:  ordem ← [ ]
3:  historico ← [ ]
4:  K ← min(K, |V|)                          ▷ evita laço infinito se K > |V|
5:  while |S| < K do
6:      critico ← NULO
7:      melhor_f ← +∞
8:      for i ← 0 até |V| − 1 do
9:          if i ∈ S then
10:             continue
11:         end if
12:         valor ← f(G, S ∪ {i})    ▷ recebe a conectividade residual se adicionar i a S
13:         if valor < melhor_f then
14:             melhor_f ← valor
15:             critico ← i
16:         end if
17:     end for
18:     S ← S ∪ {critico}
19:     ordem.ADICIONA(critico)
20:     historico.ADICIONA(melhor_f)
21:     if melhor_f = 0 then                 ▷ grafo já totalmente fragmentado
22:         break
23:     end if
24: end while
25: return (ordem, historico)
```

Implementação de nossa autoria, baseada no *Greedy2* de Addis et al. 2016.

Ventresca and Aleman 2015, mas pode ser resumida atentando a uma série de propriedades sobre os grafos e as árvores DFS derivadas deles.

A partir da árvore DFS do grafo $G = (V, E)$, observa-se, que para todo vértice $v \in V$:

- Se v é uma folha da árvore DFS, sua remoção não cria novos componentes conexos. O número de componentes do grafo residual permanece o mesmo.
- Seja w um filho de v na árvore DFS. Se a subárvore enraizada em w não possui aresta de retorno para um ancestral de v — condição verificada no Algoritmo 4 com LOW$[w] \geq$ DFN$[v]$ —, então essa subárvore se torna um componente conexo separado ao remover v, com $|t_w|$ vértices.
- Os vértices que não pertencem à subárvore de v formam, após a remoção de v, um componente com $|V(v)| - s(v) - 1$ vértices, onde $|V(v)|$ é o tamanho do componente que contém v e $s(v)$ é o número total de vértices das subárvores do item anterior, que se separam ao remover v, subtraindo uma unidade para representar a perda do próprio v.

  Combinando essas propriedades, o impacto de remover v é dado por:

$g(v) = \binom{|V(v)|-s(v)-1}{2} + \sum_{t_i \in T(v)} \binom{|t_i|}{2}$ onde $T(v)$ é o conjunto das subárvores enraizadas nos filhos w de v que satisfazem LOW$[w] \geq$ DFN$[v]$. O primeiro termo corresponde aos pares conectados na parte ancestral à v; e o segundo, aos pares internos a cada subárvore que se separa.

O pseudocódigo deste algoritmo pode ser visto em Ventresca and Aleman 2015.

A implementação efetiva foi feita com a função *EVALUATE_ALL*, detalhada no Algoritmo 4.

7.3.1. Complexidade assintótica do *EVALUATE_ALL*

A complexidade temporal de pior caso para o Algoritmo 4 é determinada pelo custo da busca em profundidade executada sobre o grafo residual $G[V \setminus S]$. Analisamos cada etapa separadamente.

A criação dos vetores auxiliares tem custo $\Theta(|V|)$. A marcação dos vértices de *removed* tem custo $\Theta(|S|) \leq O(|V|)$. O laço externo sobre as raízes percorre todos os vértices de G, executando $|V|$ iterações com custo $\Theta(|V|)$.

A análise da DFS executada no Algoritmo 4 é bastante similar àquela feita na Seção 7.2.2 sobre o Algoritmo 2. A DFS é iniciada apenas para vértices não removidos e ainda não visitados. Como a DFS marca todos os vértices de um componente como visitados, apenas o primeiro vértice não visitado de cada componente inicia uma nova DFS. Sendo T o conjunto de componentes conexos de $G[V \setminus S]$, tem-se $|T| \leq |V|$. O custo de inicialização da DFS é $O(1)$ por componente, totalizando $O(|T|) \leq O(|V|)$.

O laço *while* desempilha cada vértice uma única vez. Como cada vértice é empilhado no máximo uma vez, essas iterações ocorrem, no máximo, $|V|$ vezes, com custo $\Theta(|V|)$.

Nas demais iterações do *while*, o algoritmo examina um vizinho de v usando o vetor *next_neigh*. Como esse vetor avança monotonicamente, cada aresta é examinada no

Algorithm 4 Algoritmo *EVALUATE_ALL*: impacto de remover cada vértice.

Require: Grafo $G = (V, E)$, conjunto *removed* $\subseteq V$
Ensure: *result*$[v] = f($*removed* $\cup \{v\})$ para $v \notin$ *removed*, NULO caso contrário

1: *dfn*, *parent* $\leftarrow [-1] \times |V|$; *next*, *low*, *cut*, *split* $\leftarrow [0] \times |V|$; *subtree* $\leftarrow [1] \times |V|$
2: *is_rem* $\leftarrow [v \in$ *removed*$]$; *result* $\leftarrow$ [NULO] $\times |V|$; *total*, *num* $\leftarrow 0$
3: **for** cada *raiz* $\in V$ com $\neg$*is_rem*[*raiz*] e *dfn*[*raiz*] $= -1$ **do**
4: *dfn*[*raiz*] $\leftarrow$ *low*[*raiz*] $\leftarrow$ *num*++
5: *comp* $\leftarrow 1$; $S \leftarrow$ [*raiz*]
6: **while** $S \neq \emptyset$ **do**
7: $v \leftarrow$ TOPO(S)
8: **if** *next*$[v] < |G.$*adj*$[v]|$ **then**
9: $w \leftarrow G.$*adj*$[v]$[*next*$[v]$++]
10: **if** *is_rem*$[w]$ **ou** $w =$ *parent*$[v]$ **then continue**
11: **end if**
12: **if** *dfn*$[w] = -1$ **then**
13: *parent*$[w] \leftarrow v$; *dfn*$[w] \leftarrow$ *low*$[w] \leftarrow$ *num*++
14: *comp* $\leftarrow$ *comp* $+ 1$; EMPILHA(S, w)
15: **else**
16: *low*$[v] \leftarrow \min($*low*$[v],$ *dfn*$[w])$
17: **end if**
18: **else**
19: DESEMPILHA(S); $p \leftarrow$ *parent*$[v]$
20: **if** $p \neq -1$ **then**
21: *low*$[p] \leftarrow \min($*low*$[p],$ *low*$[v])$; *subtree*$[p] \leftarrow$ *subtree*$[p] +$ *subtree*$[v]$
22: **if** *low*$[v] \geq$ *dfn*$[p]$ **then**
23: *cut*$[p] \leftarrow$ *cut*$[p] +$ *subtree*$[v]$; *split*$[p] \leftarrow$ *split*$[p] + \binom{\textit{subtree}[v]}{2}$
24: **end if**
25: **end if**
26: **end if**
27: **end while**
28: *total* $\leftarrow$ *total* $+ \binom{\textit{comp}}{2}$
29: **for** cada v visitado nesta DFS **do**
30: *result*$[v] \leftarrow$ *split*$[v] + \binom{\textit{comp}-1-\textit{cut}[v]}{2} - \binom{\textit{comp}}{2}$
31: **end for**
32: **end for**
33: **for** cada $v \notin$ *removed* **do**
34: *result*$[v] \leftarrow$ *result*$[v] +$ *total*
35: **end for**
36: **return** *result*

Implementação de nossa autoria, baseada no *EVALUATE* de Ventresca and Aleman 2015.

máximo duas vezes: uma em cada extremidade. Portanto, o número total de exames de vizinhos é limitado por $2|E|$, com custo $\Theta(|E|)$.

Após o processamento de cada componente, os valores de *result* são calculados em $O(1)$ por vértice do componente, totalizando $\Theta(|V|)$. O laço final que soma *total_f* percorre todos os vértices uma vez, com custo também $\Theta(|V|)$.

Somando as parcelas: $\Theta(|V|)$ para a inicialização e os laços sobre vértices, e $\Theta(|E|)$ para a exploração de vizinhos; totalizamos a complexidade: $\Theta(|V| + |E|)$.

Algorithm 5 Heurística construtiva gulosa *Greedy2_dfs* com *EVALUATE_ALL*.

Require: Grafo $G = (V, E)$, inteiro K
Ensure: Lista *ordem* com os vértices removidos na ordem de escolha e lista *historico* com $f(S)$ após cada iteração

```
1: S ← ∅
2: ordem ← [ ]
3: historico ← [ ]
4: K ← min(K, |V|)                                  ▷ evita laço infinito se K > |V|
5: while |S| < K do
6:     values ← EVALUATE_ALL(G, S)   ▷ values[v] = f(S ∪ {v}) ou NULO se v ∈ S
7:     critico ← NULO
8:     melhor_f ← +∞
9:     for i ← 0 até |V| − 1 do
10:        if values[i] ≠ NULO e values[i] < melhor_f then     ▷ empates: menor índice
11:            melhor_f ← values[i]
12:            critico ← i
13:        end if
14:    end for
15:    S ← S ∪ {critico}
16:    ordem.ADICIONA(critico)
17:    historico.ADICIONA(melhor_f)
18:    if melhor_f = 0 then                          ▷ grafo já totalmente fragmentado
19:        break
20:    end if
21: end while
22: return (ordem, historico)
```

Implementação de nossa autoria, baseada no Greedy2 de Addis et al. 2016 e no *EVALUATE* de Ventresca and Aleman 2015.

7.3.2. Complexidade assintótica do *Greedy2_dfs*

A complexidade temporal de pior caso para o Algoritmo 5 é determinada pelo número de chamadas ao *EVALUATE_ALL* e pelo custo de cada chamada. Ao contrário da versão ingênua, que chama a função objetivo uma vez por vértice candidato, a versão otimizada faz uma única chamada por iteração do laço principal.

O laço *while* externo (linha 5 do Algoritmo 5) executa até que $|S| = K$, com $K \leq |V|$. Em cada iteração, o algoritmo realiza três operações: (i) uma chamada ao *EVALUATE_ALL*; (ii) um laço *for* sobre todos os vértices, para selecionar o candidato de menor valor; (iii) a inserção do vértice escolhido em S e o registro em *ordem* e *historico*.

Conforme a análise feita na Seção 7.3.1, cada chamada ao *EVALUATE_ALL* tem custo $\Theta(|V| + |E|)$.

O laço *for* interno (linha 9) percorre todos os $|V|$ vértices, executando uma comparação por vértice. O custo desta etapa é, portanto, $\Theta(|V|)$ por iteração do *while*. O *while* executa K iterações no pior caso, logo, o número total de chamadas é K.

Ao multiplicar o número de iterações do *while* pelo custo de cada iteração, obtém-se: $\Theta\big(K \cdot \big(|V| + |E| + |V|\big)\big) = \Theta\big(K \cdot \big(|V| + |E|\big)\big)$ ..

A versão ingênua possui complexidade $O(K \cdot |V| \cdot (|V| + |E|))$, logo, a otimização de Ventresca and Aleman 2015 elimina o fator $|V|$ - obtida pelo teste dos candidatos a cada iteração - reduzindo a magnitude da complexidade assintótica.

8. Aplicação dos algoritmos

Foi desenvolvida uma solução com Interface em Linha de Comando (CLI) para suportar a execução dos algoritmos implementados neste trabalho. Aplicação e os algoritmos foram implementados em linguagem Python.

A solução é projetada para receber grafos não direcionados em arquivos de texto puro no formato utilizado por Ventresca and Aleman 2015; instanciar os grafos; aplicaros algoritmos - Algoritmo 3 ou Algoritmo 5; e agregar uma saída com as informações da aplicação realizada e o tempo decorrido para execução do algoritmo.

A solução permite a execução automatizada das instâncias definidas na Seção 5.2 e a escrita em arquivo dos resultados como uma tabela em formato CSV. A tabela consta com as colunas:

- ***instancia***: Grafo dentre o conjunto de 16 entradas descritos na Seção 5.1.
- ***n***: Número de vértices do grafo.
- ***m***: Número de arestas do grafo.
- ***K***:Parâmetro de entrada.
- ***algoritmo***: Parâmetro de entrada.
- ***f_obtido***: O valor mínimo da função objetivo obtido pela execução do algoritmo na referida instância.
- ***melhor_conhecido***: O melhor valor obtido para a instância dada, extraído de Zhou et al. 2019, isto é, o menor valor entre o KBV e os resultados dos oito algoritmos da Tabela 4 daquele trabalho.
- ***gap_percent***: Diferença percentual entre o valor da função objetivo obtido pela heurística desenvolvida neste trabalho e o respectivo melhor valor conhecido, conforme mostrado por Zhou et al. 2019. A fórmula deste valor é expressa como: $\text{gap} = \frac{f_{obtido} - f_{melhor}}{f_{melhor}} \cdot 100\%$
- ***tempo_s***: Tempo decorrido em segundos para a execução da referida instância, incluída principalmente para observar a diferença de tempo de execução entre as versões ingênua e otimizada da heurística construtiva.

9. Resultados

Com a aplicação dos algoritmos mediante o método exposto na Seção 8, foram modeladas nas Tabela 3 e Tabela 2. Nota-se que os valores calculados para a função objetivo permanecem os mesmos na execução de ambos os algoritmos, incluindo também o conjunto de remoção obtido. A diferença real pode ser vista no tempo de execução das duas abordagens, com a expressiva redução na execução da versão otimizada.

Os benchmarks foram executados em máquina com as seguintes especificações: processador Intel Core i7-12700 (12ª geração, Alder Lake-S) com 12 núcleos, memória RAM com 64 GB e executando sistema operacional Ubuntu 22.04.5 LTS.

Adicionamos também a agregação dos valores obtidos pela implementação do *Greedy2* neste trabalho com os resultados apresentados por Zhou et al. 2019 na Tabela 4, permitindo a comparação com demais algoritmos da literatura. Reforçamos, como exposto na Seção 5.1, que os nomes dos grafos utilizados por Zhou et al. 2019 diferem da nomenclatura deste trabalho e que a Tabela 4 utiliza o nosso esquema de nomes.

Tabela 2. Resultados do algoritmo otimizado nas instâncias de teste.

Instância	$\|V\|$	$\|E\|$	K	f **obtido**	**Melhor conhecido**	**Gap (%)**	**Tempo (s)**
BarabasiAlbert_n500m1	500	499	50	199	195	2.05	0.021
BarabasiAlbert_n1000m1	1000	999	75	559	558	0.18	0.064
BarabasiAlbert_n2500m1	2500	2499	100	3726	3704	0.59	0.230
BarabasiAlbert_n5000m1	5000	4999	150	10 216	10 196	0.20	0.413
ErdosRenyi_n250	235	350	50	3889	295	1218.31	0.006
ErdosRenyi_n500	466	700	80	28 520	1524	1771.39	0.024
ErdosRenyi_n1000	941	1400	140	124 426	5012	2382.56	0.087
ErdosRenyi_n2500	2344	3500	200	1 393 897	902 498	54.45	0.413
ForestFire_n250	250	514	50	210	194	8.25	0.012
ForestFire_n500	500	828	110	277	257	7.78	0.032
ForestFire_n1000	1000	1817	150	1388	1260	10.16	0.102
ForestFire_n2000	2000	3413	200	4873	4545	7.22	0.468
WattsStrogatz_n250	250	1246	70	16 110	3083	422.54	0.038
WattsStrogatz_n500	500	1496	125	69 378	2072	3248.36	0.099
WattsStrogatz_n1000	1000	4996	200	319 600	109 807	191.06	0.498
WattsStrogatz_n1500	1500	4498	265	760 761	13 098	5708.22	0.660

Tabela 3. Resultados do algoritmo ingênuo nas instâncias de teste.

Instância	$\|V\|$	$\|E\|$	K	f **obtido**	**Melhor conhecido**	**Gap (%)**	**Tempo (s)**
BarabasiAlbert_n500m1	500	499	50	199	195	2.05	2.586
BarabasiAlbert_n1000m1	1000	999	75	559	558	0.18	14.745
BarabasiAlbert_n2500m1	2500	2499	100	3726	3704	0.59	105.179
BarabasiAlbert_n5000m1	5000	4999	150	10 216	10 196	0.20	617.731
ErdosRenyi_n250	235	350	50	3889	295	1218.31	0.400
ErdosRenyi_n500	466	700	80	28 520	1524	1771.39	2.545
ErdosRenyi_n1000	941	1400	140	124 426	5012	2382.56	19.379
ErdosRenyi_n2500	2344	3500	200	1 393 897	902 498	54.45	169.572
ForestFire_n250	250	514	50	210	194	8.25	0.403
ForestFire_n500	500	828	110	277	257	7.78	3.736
ForestFire_n1000	1000	1817	150	1388	1260	10.16	23.297
ForestFire_n2000	2000	3413	200	4873	4545	7.22	125.944
WattsStrogatz_n250	250	1246	70	16 110	3083	422.54	0.726
WattsStrogatz_n500	500	1496	125	69 378	2072	3248.36	3.380
WattsStrogatz_n1000	1000	4996	200	319 600	109 807	191.06	34.340
WattsStrogatz_n1500	1500	4498	265	760 761	13 098	5708.22	85.593

Tabela 4. Comparação com a Tabela 4 de Zhou et al. 2019, acrescida dos resultados do *Greedy2* implementado neste trabalho.

Instance	K	KBV	Greedy3d	Greedy4d	Best VNS	Best ILS	GA	CNA1	FastCNP	MACNP	Greedy2
BarabasiAlbert_n500m1	50	195*	195	195	195	195	195	195	195	195	199
BarabasiAlbert_n1000m1	75	558*	559	559	559	559	558	558	558	558	559
BarabasiAlbert_n2500m1	100	3704*	3722	3722	3704	3722	3704	3704	3704	3704	3726
BarabasiAlbert_n5000m1	150	10196*	10196	10196	10196	10222	10196	10196	10196	10196	10216
ErdosRenyi_n250	50	295*	315	313	295	313	295	295	295	295	3889
ErdosRenyi_n500	80	1524	1938	1993	1542	1874	1560	1524	1524	1524	28520
ErdosRenyi_n1000	140	5012	8106	8419	5198	5544	5120	5114	5012	5012	124426
ErdosRenyi_n2500	200	959500	1118785	1112685	997839	1038048	1039254	996411	953437	902498	1393897
ForestFire_n250	50	194*	199	197	194	195	194	194	194	194	210
ForestFire_n500	110	257*	262	264	257	261	257	263	257	257	277
ForestFire_n1000	150	1260*	1288	1271	1260	1276	1260	1262	1260	1260	1388
ForestFire_n2000	200	4545*	4647	4592	4549	4583	4546	4548	4546	4545	4873
WattsStrogatz_n250	70	3101	11694	11401	6610	3241	3240	3415	3085	3083	16110
WattsStrogatz_n500	125	2078	4818	11981	2130	2282	2199	2085	2072	2072	69378
WattsStrogatz_n1000	200	113638	316416	318003	139653	115914	113638	141759	123602	109807	319600
WattsStrogatz_n1500	265	13167	157621	243190	13792	14681	13662	13498	13158	13098	760761

* Valor ótimo conhecido. A coluna **Greedy2** contém os resultados obtidos neste trabalho; os valores são idênticos nas versões ingênua e com *EVALUATE*/DFS, diferindo apenas no tempo de execução, como apontado na Seção 9.

10. Conclusão

O trabalho desenvolvido cumpriu o objetivo de entender o Problema dos Nós Críticos, o estado atual da literatura e e compreender e implementar uma heurística construtiva.

A implementação do *Greedy2* exercitou conceitos referentes a conectividade de grafos; e a otimização de Ventresca and Aleman 2015 permitiu não só implementar uma rotina complexa de mapeamento de pontos de corte do grafo, como também nos deu contato prático com uma estratégia iterativa de *backtracking*.

Uma análise aprofundada dos valores agregados na Tabela 4 pode fundamentar trabalhos futuros analisando minusciosamente a relação entre as características dos grafos de entrada e as discrepâncias entre os valores de f obtidos em nossa implementação autoral e os valores destacados em outras heurísticas construtivas gulosas, como a *Greedy3d* e a *Greedy4d*.

Referências

aaaa aaaa.

Addis et al. 2016 Addis, B., Aringhieri, R., Grosso, A., and Hosteins, P. (2016). Hybrid constructive heuristics for the critical node problem. *Annals of Operations Research*, 238(1–2):637–649.

Arulselvan et al. 2009 Arulselvan, A., Commander, C. W., Elefteriadou, L., and Pardalos, P. M. (2009). Detecting critical nodes in sparse graphs. *Computers & Operations Research*, 36(7):2193–2200.

Dinh et al. 2012 Dinh, T. N., Xuan, Y., Thai, M. T., Pardalos, P. M., and Znati, T. (2012). On new approaches of assessing network vulnerability: Hardness and approximation. *IEEE/ACM Transactions on Networking*, 20(2):609–619.

Lalou et al. 2018 Lalou, M., Tahraoui, M. A., and Kheddouci, H. (2018). The critical node detection problem in networks: A survey. *Computer Science Review*, 28:92–117.

Ventresca 2012 Ventresca, M. (2012). Global search algorithms using a combinatorial unranking-based problem representation for the critical node detection problem. *Computers & Operations Research*, 39(11):2763–2775.

Ventresca and Aleman 2014 Ventresca, M. and Aleman, D. (2014). A derandomized approximation algorithm for the critical node detection problem. *Computers & Operations Research*, 43:261–270.

Ventresca and Aleman 2015 Ventresca, M. and Aleman, D. (2015). Efficiently identifying critical nodes in large complex networks. *Computational Social Networks*, 2(1):6.

Zhou et al. 2019 Zhou, Y., Hao, J.-K., and Glover, F. (2019). Memetic search for identifying critical nodes in sparse graphs. *IEEE Transactions on Cybernetics*, 49(10):3699–3712.